C++ COMPLETE REFERENCE LIBRARY • MODULE 16

16 EXCEPTION HANDLING — Complete Theory & Study Book

Comprehensive Offline Theory, Memory Diagrams, Classroom C++ Programs, Practice Questions & Solutions

Module Table of Contents

1. **Module Overview & Study Guide** (README)
2. **Theory Chapter:** 01 exceptions try throw catch fundamentals
3. **Theory Chapter:** 02 propagation stack unwinding and rethrow
4. **Theory Chapter:** 03 standard and custom exception classes
5. **Theory Chapter:** 04 raii noexcept and exception safety guarantees
6. **Visual Memory & Concept Diagrams** (2 Vector Diagrams)
7. **Annotated C++ Source Programs** (5 Programs)
8. **Practice Question Bank:** conceptual questions
9. **Practice Question Bank:** debugging questions
10. **Practice Question Bank:** predict the output
11. **Practice Question Bank:** programming questions
12. **Step-by-Step Solutions:** conceptual solutions
13. **Step-by-Step Solutions:** output solutions
14. **Step-by-Step Solutions:** programming solutions

16 — Exception Handling (Priority Deep-Dive Module)

This module provides a complete, exam-focused and standard-accurate treatment of C++ Exception Handling, including `try` - `throw` - `catch` control flow, stack unwinding, rethrowing (`throw;`), standard and custom exception hierarchies, RAII resource safety, `noexcept` , and the three formal Exception Safety Guarantees (Basic, Strong, and No-Throw).

Directory Contents

1. Theory (`01_THEORY/`)

- `01_exceptions_try_throw_catch_fundamentals.md` (`./01_THEORY/01_exceptions_try_throw_catch_fundamentals.md`) — Errors vs. exceptions, `try` , `throw` , `catch` , multiple `catch` blocks, and catch-all (`catch (...)`).
- `02_propagation_stack_unwinding_and_rethrow.md` (`./01_THEORY/02_propagation_stack_unwinding_and_rethrow.md`) — Exception propagation across call

frames, deterministic destructor invocation during **Stack Unwinding**, nested `try` blocks, and `throw;` vs `throw e;`.

- `03_standard_and_custom_exception_classes.md` (./01_THEORY/03_standard_and_custom_exception_classes.md) — `<stdexcept>` hierarchy (`std::exception`, `runtime_error`, `logic_error`, `bad_alloc`), catching by `const` reference (`catch (const exception& e)`), and writing custom classroom exception classes.
- `04_raii_noexcept_and_exception_safety_guarantees.md` (./01_THEORY/04_raii_noexcept_and_exception_safety_guarantees.md) — Resource Acquisition Is Initialization (RAII), why raw `new` leaks during exceptions, `noexcept` specifier, and the **Basic**, **Strong**, and **No-Throw** exception safety guarantees.

2. Programs (`02_PROGRAMS/`)

- `01_division_by_zero_try_catch.cpp` (./02_PROGRAMS/01_division_by_zero_try_catch.cpp) — Category A: Classroom division-by-zero exception handling using `try`, `throw`, and `catch`.
- `02_multiple_catch_and_catch_all.cpp` (./02_PROGRAMS/02_multiple_catch_and_catch_all.cpp) — Category A: Multiple typed `catch` handlers (`int`, `const char*`) and `catch (...)`.
- `03_stack_unwinding_and_rethrow.cpp` (./02_PROGRAMS/03_stack_unwinding_and_rethrow.cpp) — Category A: Observing automatic destructor calls during stack unwinding and rethrowing with `throw;`.
- `04_custom_exception_and_noexcept.cpp` (./02_PROGRAMS/04_custom_exception_and_noexcept.cpp) — Category A: Classroom `bank_account` with custom exception class and `noexcept` balance query.
- `05_uncaught_exception_runtime_error.cpp` (./02_PROGRAMS/05_uncaught_exception_runtime_error.cpp) — Category C: Runtime-error demonstration when a thrown exception has no matching `catch` block (`std::terminate`).

3. Practice Questions (`03_PRACTICE_QUESTIONS/`)

- `conceptual_questions.md` (./03_PRACTICE_QUESTIONS/conceptual_questions.md)
- `predict_the_output.md` (./03_PRACTICE_QUESTIONS/predict_the_output.md)
- `debugging_questions.md` (./03_PRACTICE_QUESTIONS/debugging_questions.md)
- `programming_questions.md` (./03_PRACTICE_QUESTIONS/programming_questions.md)

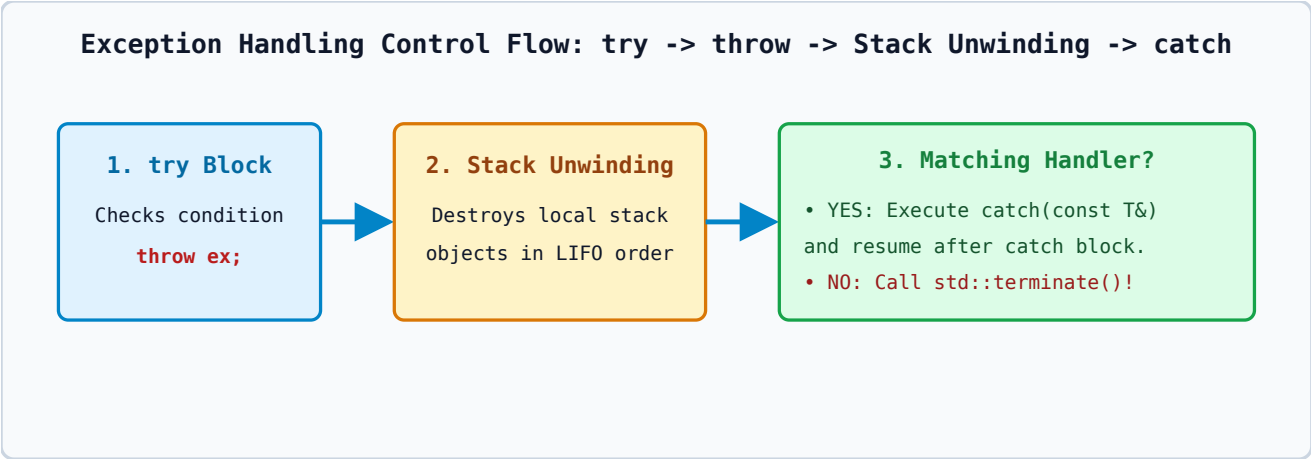
4. Detailed Solutions (`04_DETAILED_SOLUTIONS/`)

- `conceptual_solutions.md` (./04_DETAILED_SOLUTIONS/conceptual_solutions.md)
- `output_solutions.md` (./04_DETAILED_SOLUTIONS/output_solutions.md)
- `programming_solutions.md` (./04_DETAILED_SOLUTIONS/programming_solutions.md)

5. Technical Diagrams (`05_DIAGRAMS/`)

- `diagram_01_exception_handling_flowchart.svg` (./05_DIAGRAMS/diagram_01_exception_handling_flowchart.svg)
- `diagram_02_stack_unwinding_raii.svg` (./05_DIAGRAMS/diagram_02_stack_unwinding_raii.svg)

01 — Exception Handling Fundamentals: `try`, `throw`, and `catch`



Exception Handling Flowchart

1. Definition: What is an Exception?

An **exception** is a runtime anomaly or error condition (such as division by zero, out-of-bounds array index, failed memory allocation, or invalid user input) that prevents a function from completing its normal task.

Errors vs. Exceptions

- **Compile-Time (Syntax/Semantic) Errors**: Detected by the compiler before the program runs (e.g., missing semicolon, access violation). Cannot be handled with `try`-`catch`.
- **Synchronous Runtime Exceptions**: Detectable runtime conditions raised explicitly via `throw` (or by standard library functions like `vector::at()` or `new`). **Handled by C++ `try`-`catch`.**
- **Hardware Traps / Undefined Behavior**: In C++, integer division by zero (`10 / 0`) or dereferencing `nullptr` (`*p`) at the CPU level does **NOT** automatically throw a C++ exception — it is Undefined Behavior! You must explicitly check `if (denom == 0) throw ...;` to raise a C++ exception.

2. The Three Keywords: `try`, `throw`, and `catch`

CPP

```cpp
try {
    // Protected block of code where an exception might occur
    if (b == 0) {
        throw "Division by zero error!"; // Transfers control immediately to catch
    }
    cout << (a / b) << endl;
}
catch (const char* msg) {
    // Exception handler that executes only if a matching type is thrown
    cout << "Caught exception: " << msg << endl;
}
```

Strict Type-Matching Rules in `catch` (`[except.handle]`)

Unlike normal function calls, `catch` blocks do **NOT** perform implicit arithmetic conversions!

- Throwing an `int` (`throw 42;`) will **NOT** match `catch (double d)` or `catch (long l)` !
- Allowed conversions in `catch` are strictly limited to:
  1. Qualification conversions (e.g., `char*` matches `const char*`).
  2. Derived-to-Base conversions (e.g., throwing `std::runtime_error` matches `catch (const std::exception& e)`).
  3. Array-to-pointer decay (e.g., throwing `"error"` of type `const char[6]` matches `catch (const char*)`).

---

3. Multiple `catch` Blocks and Catch-All (`catch (...)`)

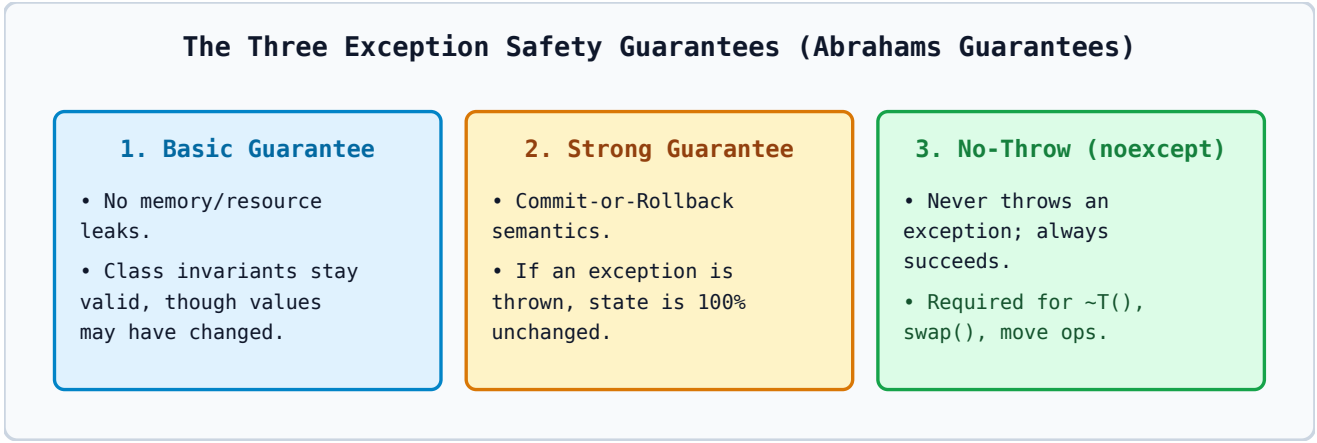
A single `try` block can be followed by multiple `catch` blocks tested **in top-to-bottom order**:

CPP

```cpp
try {
    // code
}
catch (int err_code) {
    cout << "Integer exception: " << err_code << endl;
}
catch (const char* msg) {
    cout << "String exception: " << msg << endl;
}
catch (...) {
    // Catch-all handler: catches ANY exception type not matched above
    cout << "Unknown exception caught!" << endl;
}
```

Ordering Rule: Always place **more specific (derived)** `catch` handlers **before more general (base)** `catch` handlers, and always place `catch (...)` **last**.

02 — Exception Propagation, Stack Unwinding, and Rethrowing (`throw;`)



Stack Unwinding and RAII

1. Exception Propagation Across Function Calls

If a function `funcC()` executes a `throw` statement and does not have an enclosing `try`-`catch` block inside `funcC()`, the function immediately terminates its normal execution and **propagates** the exception up the call stack to its caller `funcB()`, and if unhandled there, up to `funcA()`, and finally to `main()`.

If the exception reaches the top of `main()` without finding any matching `catch` block anywhere in the call chain, the C++ runtime invokes **`std::terminate()`**, which aborts the program (`std::abort()`).

2. What is Stack Unwinding? (`[except.ctor]`)

Stack Unwinding is the automatic process performed by the C++ runtime as an exception propagates out of a `try` block or out of nested function stack frames on its way to a matching `catch` handler:

1. Every local automatic (stack) object that was **fully constructed** inside the exited scope is automatically **destroyed** by invoking its destructor (`~ClassName()`).
2. Destructors are called in **exact reverse order of construction** (Last-In, First-Out).
3. If a constructor itself throws an exception partway through constructing an object:
   - The destructors of any **already-constructed subobjects (base classes or data members)** are automatically called.
   - The destructor of the **partially constructed object itself is NOT called** (because its construction never completed).

3. Nested `try` Blocks and Rethrowing Exceptions (`throw;` vs `throw e;`)

Sometimes an inner `catch` block wants to log an error or perform partial cleanup and then pass the **exact same exception** up to an outer `catch` block:

CPP

```cpp
try {
    try {
        throw 404;
    }
    catch (int code) {
        cout << "Inner catch logged code: " << code << endl;
        throw; // RETHROW the current exception to the outer catch block!
    }
}
catch (int code) {
    cout << "Outer catch handled rethrown code: " << code << endl;
}
```

Critical Distinction: `throw;` vs. `throw e;`

- `throw;` **(Empty throw inside a** `catch` **block)**: Rethrows the **original exception object** preserving its true dynamic polymorphic type without copying or slicing it. (Note: if `throw;` is executed when no exception is currently being handled, it immediately calls `std::terminate()` !)
- `throw e;` : Creates a **new copy** of `e` using its static type, which causes **object slicing** if `e` was caught as a base class `const std::exception&` ! Always use `throw;` to rethrow.

03 — Standard Exception Hierarchy (`<stdexcept>`) and Custom Exceptions

1. The Standard `<exception>` and `<stdexcept>` Hierarchy

All standard C++ exceptions inherit from the base class `std::exception` (`#include <exception>`), which provides a virtual member function:

```cpp
virtual const char* what() const noexcept;
```

Major Standard Exception Classes

1. **`std::logic_error`** (`<stdexcept>`): Errors detectable before execution by validating preconditions:
   - `std::invalid_argument`
   - `std::out_of_range` (thrown by `std::vector::at()` and `std::string::at()`)
   - `std::domain_error`
   - `std::length_error`
2. **`std::runtime_error`** (`<stdexcept>`): Errors detectable only at runtime:
   - `std::overflow_error`
   - `std::underflow_error`
   - `std::range_error`
3. **`std::bad_alloc`** (`<new>`): Thrown by `new` when heap allocation fails.
4. **`std::bad_cast`** (`<typeinfo>`): Thrown by `dynamic_cast<Derived&>(base_ref)` when a reference downcast fails.

2. Golden Rule: Throw by Value, Catch by `const` Reference

```cpp
try {
    throw runtime_error("Insufficient balance"); // Throw by value
}
catch (const exception& e) {                     // Catch by const reference
    cout << e.what() << endl;
}
```

Why catch by `const exception&`?

1. Avoids **object slicing** (preserves the derived class's overridden `what()` virtual function).
2. Avoids copying the exception object.

3. Avoids heap memory management issues that would occur if throwing pointers (`throw new ...`).

---

3. Writing a Simple Classroom Custom Exception Class

CPP

```cpp
class negative_amount_error {
private:
    int bad_amount;

public:
    negative_amount_error(int amt) {
        bad_amount = amt;
    }

    void show_error() const {
        cout << "Invalid negative amount: " << bad_amount << endl;
    }
};
```

Or, when integrating with the C++ standard library, inherit publicly from `std::exception` and override `const char* what() const noexcept override`.

04 — RAII, `noexcept`, and the Three Exception Safety Guarantees

1. Why Raw Pointers Leak During Stack Unwinding — And How RAII Fixes It

Consider what happens when an exception is thrown between `new` and `delete`:

CPP

```cpp
void unsafe_function() {
    int* p = new int[100]; // Allocated on heap
    throw 1;               // Exception thrown!
    delete[] p;            // NEVER EXECUTED! 100 ints leaked!
}
```

During stack unwinding, the runtime destroys local stack variables. Because `p` is a raw pointer (`int*`), destroying `p` simply pops the pointer variable off the stack — it does **not** call `delete[] p` !

Solution: RAII (Resource Acquisition Is Initialization)

Bind the resource to a stack object whose constructor acquires the resource and whose **destructor releases the resource** (such as `std::unique_ptr` , `std::vector` , or `std::fstream`). When stack unwinding destroys the stack object, its destructor automatically frees the heap memory or closes the file!

2. The Three Exception Safety Guarantees (High-Frequency University Exam Topic)

Every function in a C++ program provides one of three formal levels of **Exception Safety** (formulated by David Abrahams):

Guarantee Level	Name	Formal Definition & Guarantees	Example
Level 1	**Basic Exception Guarantee**	If an exception is thrown, **no resources are leaked** and all objects remain in a **valid, self-consistent state** (class invariants are preserved), though the exact values of the objects may have changed partway.	Modifying multiple fields in a container where invariants remain intact
Level 2	**Strong Exception Guarantee (Commit-or-Rollback)**	If an operation throws an exception, the program state is **completely unchanged** (rolled back to the exact state before the function was called — atomic "all-or-nothing" semantics).	Copy-and-swap idiom; `std::vector::push_back()`
Level 3	**No-Throw Guarantee (`noexcept`)**	The operation is guaranteed **never to throw an exception** and always succeeds.	Destructors (`~T()`), `swap()`, move constructors (`noexcept`), simple getters

*(Note: A function that leaks memory or corrupts invariants when an exception occurs offers **No Exception Guarantee**.)*

---

3. The `noexcept` Specifier and Destructor Rule

CPP

```cpp
int get_balance() const noexcept {
    return balance;
}
```

1. Marking a function `noexcept` tells both the human reader and the C++ compiler that the function guarantees not to throw any exceptions.
2. **What happens if a `noexcept` function throws anyway?**: The runtime does **not** propagate the exception to callers; it immediately calls `std::terminate()`.
3. **Destructors are implicitly `noexcept` since C++11**: Never let an exception escape from a destructor! If a destructor throws an exception while stack unwinding is already in progress from another exception, C++ immediately calls `std::terminate()`.

Visual Concept & Memory Diagrams (16 EXCEPTION HANDLING)

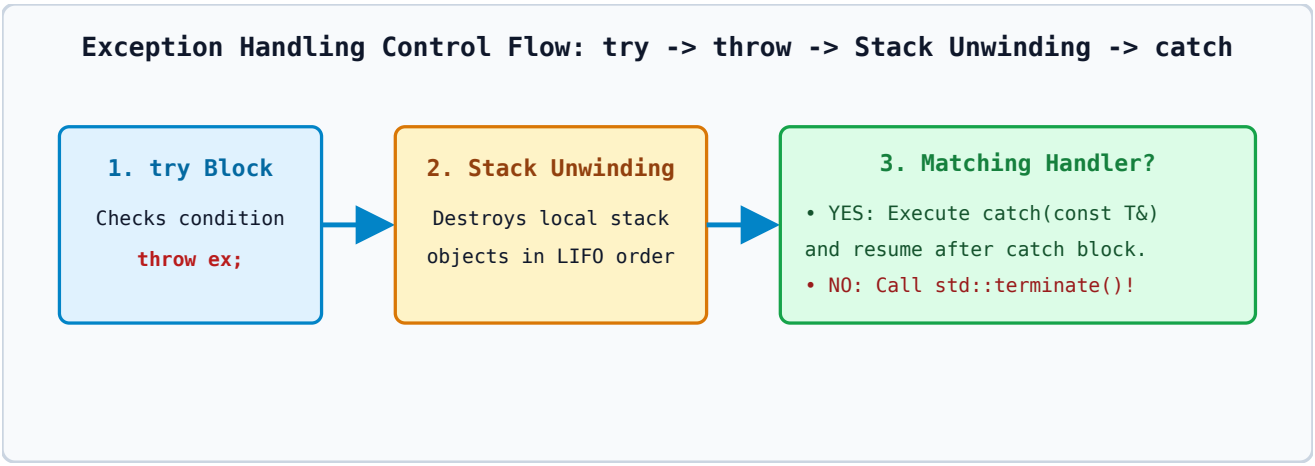


Diagram: diagram 01 exception handling flowchart

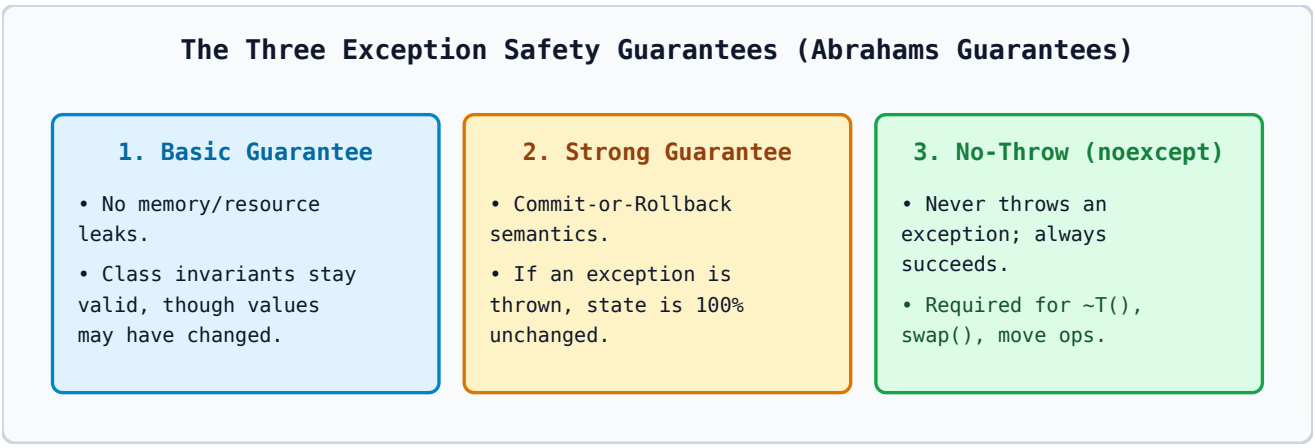


Diagram: diagram 02 stack unwinding raii

Annotated C++ Programs (16 EXCEPTION HANDLING)

Program: `01_division_by_zero_try_catch.cpp`

C++ SOURCE FILE • 01_division_by_zero_try_catch.cpp

```cpp
// Program Title: Division by Zero Exception Handling Using try, throw, and catch
// Topic: Exception Handling — Basic try-throw-catch Flow
// Purpose: Demonstrate checking a division precondition and throwing an exception caught in main().
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/01_exceptions_try_throw_catch_fundamentals.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

int divide(int num, int den) {
    if (den == 0) {
        throw "Error: Division by zero is not allowed!";
    }
    return num / den;
}

int main() {
    try {
        cout << "Result 1: " << divide(20, 4) << endl;
        cout << "Result 2: " << divide(15, 0) << endl;
        cout << "This line is skipped after throw." << endl;
    }
    catch (const char* msg) {
        cout << "Caught exception: " << msg << endl;
    }

    cout << "Program continues normally after catch." << endl;
    return 0;
}

/*
Expected Deterministic Output:
Result 1: 5
Caught exception: Error: Division by zero is not allowed!
Program continues normally after catch.
*/
```

Program: `02_multiple_catch_and_catch_all.cpp`

C++ SOURCE FILE • 02_multiple_catch_and_catch_all.cpp

```cpp
// Program Title: Multiple catch Blocks and Catch-All Handler (catch (...))
// Topic: Exception Handling — Multi-Type Exception Dispatch
// Purpose: Show how different exception types match specific catch handlers and how
catch(...) handles unknown types.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/01_exceptions_try_throw_catch_fundamentals.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

void test_exception(int mode) {
    try {
        if (mode == 1) {
            throw 404;
        } else if (mode == 2) {
            throw "Invalid access token";
        } else if (mode == 3) {
            throw 3.14;
        }
    }
    catch (int code) {
        cout << "Caught int code: " << code << endl;
    }
    catch (const char* text) {
        cout << "Caught C-string: " << text << endl;
    }
    catch (...) {
        cout << "Caught unknown exception via catch(...)" << endl;
    }
}

int main() {
    test_exception(1);
    test_exception(2);
    test_exception(3);
    return 0;
}

/*
Expected Deterministic Output:
Caught int code: 404
Caught C-string: Invalid access token
Caught unknown exception via catch(...)
*/
```

Program: `03_stack_unwinding_and_rethrow.cpp`

C++ SOURCE FILE • 03_stack_unwinding_and_rethrow.cpp

```cpp
// Program Title: Stack Unwinding Destructor Order and Rethrowing (throw;)
// Topic: Exception Handling — Stack Unwinding and Rethrowing
// Purpose: Demonstrate that local objects are destroyed in reverse order during stack
unwinding and how throw; rethrows.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/02_propagation_stack_unwinding_and_rethrow.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

class resource_guard {
private:
    int id;

public:
    resource_guard(int i) {
        id = i;
        cout << "Constructed resource_guard " << id << endl;
    }

    ~resource_guard() {
        cout << "Destroyed resource_guard " << id << " during stack unwinding" << endl;
    }
};

void risky_operation() {
    resource_guard g1(1);
    resource_guard g2(2);
    cout << "Throwing exception inside risky_operation..." << endl;
    throw 99;
}

int main() {
    try {
        try {
            risky_operation();
        }
        catch (int err) {
            cout << "Inner catch received " << err << ", rethrowing..." << endl;
            throw; // Rethrows original exception to outer catch
        }
    }
    catch (int err) {
        cout << "Outer catch handled " << err << endl;
    }

    return 0;
}

/*
Expected Deterministic Output:
Constructed resource_guard 1
```

```
Constructed resource_guard 2
Throwing exception inside risky_operation...
Destroyed resource_guard 2 during stack unwinding
Destroyed resource_guard 1 during stack unwinding
Inner catch received 99, rethrowing...
Outer catch handled 99
*/
```

Program: `04_custom_exception_and_noexcept.cpp`

C++ SOURCE FILE • 04_custom_exception_and_noexcept.cpp

```cpp
// Program Title: Custom Classroom Exception Class and noexcept Member Function
// Topic: Exception Handling — Custom Exceptions, Strong Guarantee, and noexcept
// Purpose: Demonstrate a custom exception class in a classroom bank_account with Strong
Guarantee withdrawal.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/03_standard_and_custom_exception_classes.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

class insufficient_funds_error {
private:
    int requested;
    int available;

public:
    insufficient_funds_error(int req, int avail) {
        requested = req;
        available = avail;
    }

    void show_error() const noexcept {
        cout << "Exception: Requested " << requested
             << ", but available balance is only " << available << endl;
    }
};

class bank_account {
private:
    int balance;

public:
    bank_account(int initial) {
        balance = initial;
    }

    // Strong Exception Guarantee: if withdrawal fails, balance remains completely unchanged!
    void withdraw(int amount) {
        if (amount > balance) {
            throw insufficient_funds_error(amount, balance);
        }
        balance = balance - amount;
    }

    int get_balance() const noexcept {
        return balance;
    }
};

int main() {
    bank_account acc(1000);
```

```
    try {
        acc.withdraw(400);
        cout << "Balance after 400 withdrawal: " << acc.get_balance() << endl;
        acc.withdraw(800); // Throws insufficient_funds_error
    }
    catch (const insufficient_funds_error& err) {
        err.show_error();
    }

    cout << "Balance remains unchanged after failed withdrawal: " << acc.get_balance() <<
endl;
    return 0;
}

/*
Expected Deterministic Output:
Balance after 400 withdrawal: 600
Exception: Requested 800, but available balance is only 600
Balance remains unchanged after failed withdrawal: 600
*/
```

Program: `05_uncaught_exception_runtime_error.cpp`

C++ SOURCE FILE • 05_uncaught_exception_runtime_error.cpp

```cpp
// Program Title: Uncaught Exception Resulting in Runtime Termination (std::terminate)
// Topic: Exception Handling — Type Mismatch in catch Block
// Purpose: Show how throwing an 'int' when only 'catch (double)' is present causes runtime termination.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/01_exceptions_try_throw_catch_fundamentals.md
// Program Classification: Category C — Runtime-error demonstration

#include <iostream>
using namespace std;

int main() {
    try {
        // Throws an 'int' (100).
        // Because C++ exception handling does NOT perform implicit int-to-double promotion,
        // 'catch (double)' below does NOT match!
        throw 100;
    }
    catch (double d) {
        cout << "Caught double: " << d << endl;
    }

    return 0;
}

/*
Runtime-Error Explanation (Category C):
1. This program compiles cleanly with zero syntax errors.
2. At runtime, 'throw 100;' throws an exception of type 'int'.
3. The handler 'catch (double d)' requires an exact match for 'double' (no arithmetic conversion from 'int' to 'double' is performed during catch matching).
4. Because the exception goes uncaught out of 'main()', the C++ runtime invokes 'std::terminate()', which aborts execution.
*/
```

Conceptual Questions — Module 16: Exception Handling

Q1. Differentiate between compile-time errors, synchronous runtime exceptions, and hardware traps/Undefined Behavior in C++.

Q2. Explain **Stack Unwinding** in detail. In what order are local stack objects destroyed when an exception is thrown inside a `try` block? What happens if an object's constructor throws an exception halfway through its body?

Q3. What is the difference between rethrowing an exception using `throw;` versus `throw e;` inside a `catch (const BaseException& e)` block?

Q4. Define and compare the **Basic**, **Strong**, and **No-Throw (`noexcept`)** exception safety guarantees with an example of each. Why must destructors in C++ never throw exceptions?

Debugging Questions — Module 16: Exception Handling

Q7. Identify the handler-ordering bug in the following `try` - `catch` sequence:

CPP

```cpp
#include <iostream>
#include <stdexcept>
using namespace std;

int main() {
    try {
        throw out_of_range("Index out of bounds");
    }
    catch (const exception& e) {
        cout << "General exception\n";
    }
    catch (const out_of_range& e) {
        cout << "Out of range exception\n";
    }
    return 0;
}
```

Q8. Identify the memory leak when an exception is thrown in the following function and show how to fix it using RAII (`std::unique_ptr`):

CPP

```cpp
void process(int x) {
    int* buffer = new int[50];
    if (x < 0) {
        throw "Negative input";
    }
    delete[] buffer;
}
```

Predict the Output Questions — Module 16: Exception Handling

Q5. Predict the output:

CPP

```cpp
#include <iostream>
using namespace std;

class item {
    int id;
public:
    item(int i) : id(i) { cout << "C" << id << " "; }
    ~item() { cout << "D" << id << " "; }
};

int main() {
    try {
        item a(1);
        item b(2);
        throw 10;
        item c(3);
    }
    catch (int x) {
        cout << "Caught:" << x << " ";
    }
    cout << "End" << endl;
    return 0;
}
```

Q6. Predict the output (Base vs. Derived exception catching by reference):

CPP

```cpp
#include <iostream>
using namespace std;

class base_err {
public:
    virtual void msg() const { cout << "BaseError\n"; }
};

class derived_err : public base_err {
public:
    void msg() const override { cout << "DerivedError\n"; }
};

int main() {
    try {
        throw derived_err();
    }
    catch (const base_err& e) {
        e.msg();
    }
    return 0;
}
```

Programming Questions — Module 16: Exception Handling

Q9. Write a C++23 classroom program with a `student_marks` class that validates whether entered marks are in the range `[0, 100]`. Throw an integer exception `-1` if marks are negative and `+1` if marks exceed `100`, and handle the exception in `main()`.

Q10. Write a C++23 classroom program demonstrating a `safe_array` class of 5 integers whose `get(int index)` member function throws a custom `index_error` object when `index < 0` or `index ⩾ 5`.

Detailed Solutions — Conceptual Questions (Module 16: Exception Handling)

Solution to Q1 (Compile Errors vs. Exceptions vs. Undefined Behavior)

- **Compile-time errors**: Syntax or type/access violations rejected by the compiler prior to execution.
- **Synchronous runtime exceptions**: Runtime conditions reported explicitly via `throw` and caught via `catch`.
- **Hardware traps / Undefined Behavior**: Operations like raw `1 / 0` or `*nullptr` do **not** throw C++ exceptions; you must explicitly check preconditions (`if (den == 0) throw ...;`) to enter the C++ exception mechanism.

Solution to Q2 (Stack Unwinding & Constructor Exceptions)

- During **Stack Unwinding**, all fully constructed automatic (stack) objects in the exited `try` block and intermediate call frames are destroyed in **reverse order of construction (LIFO)**.
- If a constructor throws an exception before finishing, destructors are run for all **already-constructed base classes and member subobjects**, but the destructor for the partially constructed object itself is **not** called.

Solution to Q3 (`throw;` vs `throw e;`)

- `throw;` rethrows the original in-flight exception object without copying or slicing it.
- `throw e;` copies `e` using its static parameter type (`BaseException`), slicing off any `DerivedException` part!

Solution to Q4 (Basic, Strong, and No-Throw Guarantees & Destructors)

- **Basic**: No leaks; invariants remain valid, though state may have partially changed.
- **Strong**: Commit-or-rollback; state is completely unchanged if an exception occurs.
- **No-Throw (`noexcept`)**: Never throws; always succeeds.
- **Why destructors must not throw**: Destructors are called during stack unwinding while an exception is already active. If a destructor throws a second unhandled exception during stack unwinding, C++ cannot propagate two exceptions simultaneously and immediately calls `std::terminate()`.

Detailed Solutions — Predict the Output & Debugging (Module 16: Exception Handling)

Solution to Q5

- `item a(1)` prints `C1 `.
- `item b(2)` prints `C2 `.
- `throw 10;` triggers stack unwinding before `item c(3)` is reached: `b` is destroyed first (`D2 `), then `a` is destroyed (`D1 `).
- `catch (int x)` prints `Caught:10 `, followed by `End `.
- **Guaranteed Output**:

```
TEXT
C1 C2 D2 D1 Caught:10 End
```

Solution to Q6

- Because `e` is caught by `const` reference (`const base_err& e`) and `msg()` is `virtual`, dynamic dispatch calls `derived_err::msg()`.
- **Guaranteed Output**:

```
TEXT
DerivedError
```

Solution to Q7 (Debugging Handler Ordering)

- **Bug**: `std::out_of_range` inherits from `std::exception`. Because `catch (const exception& e)` appears **first**, it catches every `out_of_range` exception and makes the second `catch (const out_of_range& e)` unreachable dead code!
- **Fix**: Place `catch (const out_of_range& e)` **before** `catch (const exception& e)`.

Solution to Q8 (Debugging Memory Leak on `throw`)

- **Bug**: When `x < 0`, `throw "Negative input";` exits `process()` before `delete[] buffer;` is reached, leaking 50 `int`s on the heap.
- **Fix**: Use `std::unique_ptr<int[]> buffer = std::make_unique<int[]>(50);` (or `std::vector<int> buffer(50);`) so stack unwinding frees the memory automatically.

Detailed Solutions — Programming Questions (Module 16: Exception Handling)

Solution to Q9 (`student_marks` Range Validation)

CPP

```cpp
#include <iostream>
using namespace std;

class student_marks {
private:
    int marks;

public:
    void input(int m) {
        if (m < 0) {
            throw -1;
        }
        if (m > 100) {
            throw 1;
        }
        marks = m;
    }

    void display() const {
        cout << "Valid Marks: " << marks << endl;
    }
};

int main() {
    student_marks s;

    try {
        s.input(85);
        s.display();
        s.input(120); // Throws +1
    }
    catch (int code) {
        if (code == -1) {
            cout << "Error: Marks cannot be negative!" << endl;
        } else if (code == 1) {
            cout << "Error: Marks cannot exceed 100!" << endl;
        }
    }

    return 0;
}
/*
Expected Output:
Valid Marks: 85
Error: Marks cannot exceed 100!
*/
```

Solution to Q10 (`safe_array` with Custom `index_error`)

CPP

```cpp
#include <iostream>
using namespace std;

class index_error {
private:
    int bad_idx;

public:
    index_error(int idx) {
        bad_idx = idx;
    }

    void show() const {
        cout << "Index out of bounds: " << bad_idx << endl;
    }
};

class safe_array {
private:
    int arr[5] = {10, 20, 30, 40, 50};

public:
    int get(int index) const {
        if (index < 0 || index >= 5) {
            throw index_error(index);
        }
        return arr[index];
    }
};

int main() {
    safe_array sa;

    try {
        cout << "Element at 2: " << sa.get(2) << endl;
        cout << "Element at 7: " << sa.get(7) << endl;
    }
    catch (const index_error& err) {
        err.show();
    }

    return 0;
}
/*
Expected Output:
Element at 2: 30
Index out of bounds: 7
*/
```